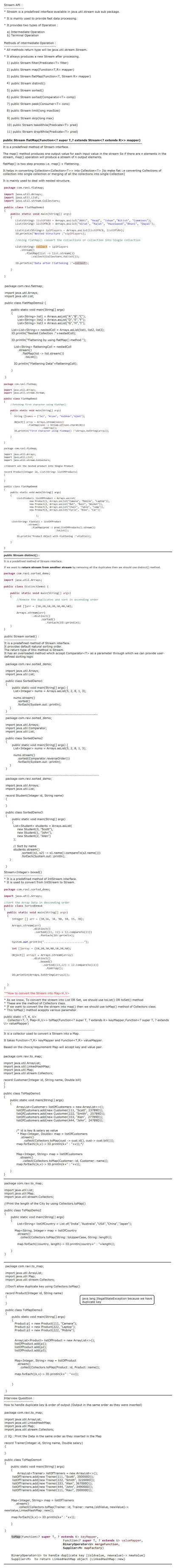

Stream API :

* Stream is a predefined interface available in java.util.stream sub sub package.

* It is mainly used to provide fast data processing.

* It provides two types of Operation :

  a) Intermediate Operation
  b) Terminal Operation

Methods of intermediate Operation :

* All methods return type will be java.util.stream.Stream.

* It always produces a new Stream after processing.

  1) public Stream filter(Predicate<T> filter)

  2) public Stream map(Function<T,R> mapper)

  3) public Stream flatMap(Function<T, Stream R> mapper)

  4) public Stream distinct()

  5) public Stream sorted()

  6) public Stream sorted(Comparator<T> comp)

  7) public Stream peek(Consumer<T> cons)

  8) public Stream limit(long maxSize)

  9) public Stream skip(long max)

  10) public Stream takeWhile(Predicate<T> pred)

  11) public Stream dropWhile(Predicate<T> pred)

public Stream flatMap(Function<? super T,? extends Stream<? extends R>> mapper)

It is a predefined method of Stream interface.

The map() method produces one output value for each input value in the stream So if there are n elements in the stream, map() operation will produce a stream of n output elements.

flatMap() is two step process i.e. map() + Flattening.

It helps in converting Collection<Collection<T>> into Collection<T> [to make flat i.e converting Collections of collection into single collection or merging of all the collections into single Collection]

It is mainly used to deal with nested structure.

```
package com.ravi.flatmap;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class FlatMapDemo1
{
    public static void main(String[] args)
    {
       List<String> listOfSRH = Arrays.asList("Abhi", "Head", "Ishan","Nitish", "Cummines");
       List<String> listOfRCB = Arrays.asList("Virat", "Rajat", "Hazelwood","Bhuvi", "Dayal");

       List<List<String>> iplPlayers = Arrays.asList(listOfRCB, listOfSRH);
       IO.println("Nested Structure :"+iplPlayers);

       //Using flatMap() convert the Collections of collection into Single collection

       List<String> collect = iplPlayers
          .stream()
             .flatMap(list -> list.stream())
                 .collect(Collectors.toList());

       IO.println("Data after Flattening :"+collect);

    }

}
```

```
package com.ravi.flatmap;

import java.util.Arrays;
import java.util.List;

public class FlatMapDemo2 {

    public static void main(String[] args)
    {
        List<String> list1 = Arrays.asList("A","B","C");
        List<String> list2 = Arrays.asList("D","E","F");
        List<String> list3 = Arrays.asList("G","H","I");

    List<List<String>> nestedColl = Arrays.asList(list1, list2, list3);
    IO.println("Nested Collection :"+nestedColl);

    IO.println("Flattening by using flatMap() method ");

      List<String> flatteningColl = nestedColl
         .stream()
            .flatMap(list -> list.stream())
              .toList();

      IO.println("Flattening Data"+flatteningColl);

    }

}
```

```
package com.ravi.flatmap;

import java.util.Arrays;
import java.util.stream.Stream;

public class FlatMapDemo3
{
    //Fetching first character using flatMap()

    public static void main(String[] args)
    {
       String []names = {"Jai", "Aryan", "Vaibhav","Ajeet"};

       Object[] array = Arrays.stream(names)
                .flatMap(name -> Stream.of(name.charAt(0)))
                          .toArray();
       IO.println("First Character using flatmap() :"+Arrays.toString(array));

    }

}
```

```
package com.ravi.flatmap;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

//Convert all the nested product into Single Product

record Product(Integer id, List<String> listOfProducts)
{

}

public class FlatMapDemo4
{
    public static void main(String[] args)
    {
       List<Product> listOfProduct = Arrays.asList(
                new Product(1, Arrays.asList("Camera", "Mobile", "Laptop")),
                new Product(2, Arrays.asList("Bat", "Ball", "Wicket")),
                new Product(3, Arrays.asList("Chair", "Table", "Lamp")),
                new Product(4, Arrays.asList("Cycle", "Bike", "Car"))

                    );

     List<String> flatColl = listOfProduct
               .stream()
                 .flatMap(prod -> prod.listOfProducts().stream())
                               .toList();

       IO.println("Product Object with flattening :"+flatColl);

    }

}
```

---

public Stream distinct() :

It is a predefined method of Stream interface.

If we want to **return stream from another stream** by removing all the duplicates then we should use distinct() method.

```
package com.ravi.sorted_demo;

import java.util.Arrays;

public class DistinctDemo1 {

    public static void main(String[] args)
    {
        //Remove the duplicates and sort in ascending order

        int []arr = {10,20,10,20,30,40,50};

        Arrays.stream(arr)
                  .distinct()
                       .sorted()
                          .forEach(IO::println);
    }
}
```

public Stream sorted() :

It is a predefined method of Stream interface.
It provides default natural sorting order.
The return type of this method is Stream.
It has an overloaded method which accept Comparator<T> as a parameter through which we can provide user-defined sorting logic

```
package com.ravi.sorted_demo;

import java.util.Arrays;
import java.util.List;

public class SortedDemo1
{
    public static void main(String[] args) {
     List<Integer> nums = Arrays.asList(5, 2, 8, 1, 3);

     nums.stream()
        .sorted()
        .forEach(System.out::println);
   }
}
```

```
package com.ravi.sorted_demo;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class SortedDemo2
{
    public static void main(String[] args) {
     List<Integer> nums = Arrays.asList(5, 2, 8, 1, 3);

     nums.stream()
        .sorted(Comparator.reverseOrder())
        .forEach(System.out::println);
   }
}
```

```
package com.ravi.sorted_demo;

import java.util.Arrays;
import java.util.List;

record Student(Integer id, String name)
{

}

public class SortedDemo3
{
    public static void main(String[] args)
    {
     List<Student> students = Arrays.asList(
        new Student(3, "Scott"),
        new Student(1, "John"),
        new Student(2, "Alen")
     );

     // Sort by name
     students.stream()
           .sorted((s1, s2) -> s1.name().compareTo(s2.name()))
           .forEach(System.out::println);
   }

}
```

Stream<Integer> boxed()

* It is a predefined method of IntStream interface.
* It is used to convert from IntStream to Stream.

```
package com.ravi.sorted_demo;

import java.util.Arrays;

//Sort the Array Data in descending order
public class SortedDemo4
{
  public static void main(String[] args)
  {
    Integer [] arr = {10,56, 34, 90, 10, 15, 78};

    Arrays.stream(arr)
                .distinct()
                 .sorted((i1, i2)-> i2.compareTo(i1))
                    .forEach(IO::println);

    System.out.println(".........................");

    int []array = {10,20,30,40,10,20,50};

    Object[] array2 = Arrays.stream(array)
                .distinct()
                    .boxed()
                      .sorted((i1,i2)-> i2.compareTo(i1))
                        .toArray();

    IO.println(Arrays.toString(array2));


  }
}
```

**How to convert the Stream into Map<K,V>

* As we know, To convert the stream into List OR Set, we should use toList() OR toSet() method
* These are the method of Collectors class.
* If we want to convert the the stream into map() then we should use toMap() method of Collectors class.
* This toMap() method aceppts various parameter.

```
public static <T, K, U>
   Collector<T, ?, Map<K,U>> toMap(Function<? super T, ? extends K> keyMapper,Function<? super T, ? extends U> valueMapper)
```

It is a collector used to convert a Stream into a Map.

It takes Function<T,R> keyMapper and Function<T,R> valueMapper.

Based on the choice/requirement Map will accept key and value pair.

```
package com.ravi.to_map;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.stream.Collectors;

record Customer(Integer id, String name, Double bill)
{
}

public class ToMapDemo1
{
    public static void main(String[] args)
    {
        ArrayList<Customer> listOfCustomers = new ArrayList<>();
        listOfCustomers.add(new Customer(111, "Scott", 23789D));
        listOfCustomers.add(new Customer(222, "Smith",  25789D));
        listOfCustomers.add(new Customer(333, "Alen",  27789D));
        listOfCustomers.add(new Customer(444, "John",  24789D));


        /* id is key & salary as value
         * Map<Integer, Double> map = listOfCustomers
           .stream()
              .collect(Collectors.toMap(cust -> cust.id(), cust-> cust.bill()));
        map.forEach((k,v)-> IO.println(k+" : "+v));*/


        Map<Integer, String> map = listOfCustomers
          .stream()
             .collect(Collectors.toMap(Customer::id, Customer::name));
        map.forEach((k,v)-> IO.println(k+" : "+v));

    }
}
```

```
package com.ravi.to_map;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

//Print the length of the City by using Collectors.toMap()

public class ToMapDemo2
{
    public static void main(String[] args)
    {
        List<String> listOfCountry = List.of("India","Australia","USA","China","Japan");

      Map<String, Integer> map = listOfCountry
        .stream()
          .collect(Collectors.toMap(String::toUpperCase, String::length));

        map.forEach((country, length)-> IO.println(country+" : "+length));

    }
}
```

```
package com.ravi.to_map;

import java.util.ArrayList;
import java.util.Map;
import java.util.stream.Collectors;

//Don't allow duplicate key using Collectors.toMap()

record Product(Integer id, String name)
{

}

public class ToMapDemo3
{
    public static void main(String[] args)
    {
      Product p1 = new Product(111, "Camera");
      Product p2 = new Product(222, "Laptop");
      Product p3 = new Product(222, "Mobile");


      ArrayList<Product> listOfProduct = new ArrayList<>();
      listOfProduct.add(p1);
      listOfProduct.add(p2);
      listOfProduct.add(p3);


      Map<Integer, String> map = listOfProduct
        .stream()
          .collect(Collectors.toMap(Product::id, Product::name));

      map.forEach((k,v)-> IO.println(k+" : "+v));


    }

}
```

java.lang.IllegalStateException because we have duplicate key

---

Interview Questtion :

How to handle duplicate key & order of output (Output in the same order as they were inserted)

```
package com.ravi.to_map;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.stream.Collectors;

// IQ : Print the Data in the same order as they inserted in the Map

record Trainer(Integer id, String name, Double salary)
{

}

public class ToMapDemo4
{
    public static void main(String[] args)
    {
        ArrayList<Trainer> listOfTrainers = new ArrayList<>();
     listOfTrainers.add(new Trainer(111, "Scott", 350000D));
     listOfTrainers.add(new Trainer(222, "Smith", 322000D));
     listOfTrainers.add(new Trainer(333, "Alen", 367000D));
     listOfTrainers.add(new Trainer(444, "John", 349000D));
     listOfTrainers.add(new Trainer(111, "Ravi", 350000D));


    Map<Integer, String> map = listOfTrainers
       .stream()
          .collect(Collectors.toMap(Trainer::id, Trainer::name,(oldValue, newValue)->
newValue,LinkedHashMap::new));

    map.forEach((k,v)-> IO.println(k+" : "+v));

    }

}
```

```
    toMap(Function<? super T, ? extends K> keyMapper,
                                 Function<? super T, ? extends U> valueMapper,
                                 BinaryOperator<U> mergeFunction,
                                 Supplier<M> mapFactory)

    BinaryOperator<U> to handle duplicate key [(oldvalue, newvalue)-> newValue]
    Supplier<M>  to return LinkedHashMap object (LinkedHashMap::new)
```
